17 — MLOps & Experiment Tracking

Time: ~4-5 hours | **Level**: Professional

What you'll learn:

- MLflow: experiment tracking, model registry, deployment
- Data drift detection: PSI and statistical monitoring
- A/B testing for ML models: statistical significance
- ML pipelines: automating train → evaluate → deploy
- Weights & Biases: collaborative experiment tracking

Prerequisites: Notebook 10 (Production fine-tuning), Notebook 16 (Evaluation)

MLOps = DevOps for Machine Learning

The model is 5% of the work. MLOps handles the other 95%: versioning, reproducibility, monitoring, retraining, rollback.

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json
import time
from datetime import datetime, timedelta

sns.set_theme(style='whitegrid', font_scale=1.1)
np.random.seed(42)
```

1. Experiment Tracking — Why and How

The problem without tracking:

```
model_v2_final.pt
model_v2_final_REAL.pt
model_v2_final_REAL_fixed.pt
model_v3_lr0001_bs32_do05.pt  ← Which hyperparameters gave best results?
```

What to track:

- **Parameters**: learning_rate, batch_size, model_arch, optimizer
- **Metrics**: train_loss, val_loss, accuracy, F1, AUC (per epoch)

- **Artifacts**: model weights, predictions, plots
- **Code version**: git commit hash
- **Data version**: dataset hash or DVC reference

In [ ]:

```
# ── MLflow experiment tracking ──────────────────────────────

try:
    import mlflow

    mlflow.set_tracking_uri("sqlite:///mlflow.db")
    mlflow.set_experiment("model-comparison")

    # Simulate training runs
    configs = [
        {"lr": 0.001, "batch_size": 32, "model": "resnet18", "dropout": 0.3}
        {"lr": 0.0005, "batch_size": 64, "model": "resnet34", "dropout": 0.5
        {"lr": 0.001, "batch_size": 32, "model": "resnet50", "dropout": 0.2}
    ]

    for config in configs:
        with mlflow.start_run():
            # Log parameters
            mlflow.log_params(config)

            # Simulate training
            for epoch in range(10):
                train_loss = 2.0 * np.exp(-epoch/3) + np.random.normal(0, 0.
                val_loss = 2.0 * np.exp(-epoch/4) + np.random.normal(0, 0.08
                mlflow.log_metrics({
                    "train_loss": train_loss,
                    "val_loss": val_loss,
                }, step=epoch)

            # Log final metrics
            final_acc = 0.85 + np.random.uniform(0, 0.1)
            mlflow.log_metric("final_accuracy", final_acc)

            print(f"Run: {config['model']} lr={config['lr']} → acc={final_ac

    print("\nMLflow UI: mlflow ui --port 5000")

except ImportError:
    print("MLflow not installed: pip install mlflow")
    print("\nManual tracking example below:")

# Manual tracking (always works)
class ExperimentTracker:
    def __init__(self):
        self.runs = []

    def log_run(self, params, metrics):
        self.runs.append({
            "timestamp": datetime.now().isoformat(),
            "params": params,
            "metrics": metrics,
```

```
        })

    def best_run(self, metric="accuracy"):
        return max(self.runs, key=lambda r: r["metrics"].get(metric, 0))

    def summary(self):
        df = pd.DataFrame([
            {**r["params"], **r["metrics"]}
            for r in self.runs
        ])
        return df

tracker = ExperimentTracker()
for config in configs:
    tracker.log_run(config, {"accuracy": 0.85 + np.random.uniform(0, 0.1)})

print("\nExperiment summary:")
print(tracker.summary().to_string(index=False))
```

2. Model Registry — Version Control for Models

Model lifecycle:

```
Development → Staging → Production → Archived
     ↓           ↓          ↓
  Experiment   Shadow test  Live traffic
```

What to store:

- Model artifacts (weights)
- Training metadata (date, dataset, metrics)
- Dependencies (requirements.txt)
- Version number + promotion history

In [ ]:
```
# ── Model Registry implementation ──────────────────────────────

class ModelRegistry:
    """Simple model registry for tracking model versions."""

    STAGES = ['development', 'staging', 'production', 'archived']

    def __init__(self):
        self.models = {}

    def register(self, name, version, metrics, stage='development'):
        key = f"{name}/v{version}"
        self.models[key] = {
            'name': name,
            'version': version,
            'metrics': metrics,
            'stage': stage,
            'registered_at': datetime.now().isoformat(),
```

```
            'history': [{'stage': stage, 'timestamp': datetime.now().isoform
        }
        print(f"Registered {key} → {stage}")

    def promote(self, name, version, new_stage):
        key = f"{name}/v{version}"
        if key not in self.models:
            raise ValueError(f"Model {key} not found")
        old_stage = self.models[key]['stage']
        self.models[key]['stage'] = new_stage
        self.models[key]['history'].append({
            'stage': new_stage,
            'timestamp': datetime.now().isoformat()
        })
        print(f"Promoted {key}: {old_stage} → {new_stage}")

    def get_production_model(self, name):
        for key, model in self.models.items():
            if model['name'] == name and model['stage'] == 'production':
                return model
        return None

    def summary(self):
        rows = []
        for key, model in self.models.items():
            rows.append({
                'model': key,
                'stage': model['stage'],
                **model['metrics'],
            })
        return pd.DataFrame(rows)

# Demo
registry = ModelRegistry()
registry.register("fraud-detector", 1, {"accuracy": 0.92, "auc": 0.95})
registry.register("fraud-detector", 2, {"accuracy": 0.94, "auc": 0.97})
registry.register("fraud-detector", 3, {"accuracy": 0.93, "auc": 0.96})

registry.promote("fraud-detector", 2, "staging")
registry.promote("fraud-detector", 2, "production")
registry.promote("fraud-detector", 1, "archived")

print("\nModel Registry:")
print(registry.summary().to_string(index=False))
```

3. Data Drift Detection

Types of drift:

Type	What Changes	Detection
Data drift	Input distribution P(X)	PSI, KS test
Concept drift	Relationship P(Y|X)	Performance monitoring

Type	What Changes	Detection
Label drift	Output distribution P(Y)	Label distribution tracking

PSI (Population Stability Index):

- PSI < 0.1: No significant change
- 0.1 < PSI < 0.25: Moderate change (investigate)
- PSI > 0.25: Significant change (retrain!)

In [ ]:
```
# ——— Data drift detection with PSI ————————————————————————

def calculate_psi(reference, current, bins=10):
    """Population Stability Index for drift detection."""
    # Create bins from reference data
    breakpoints = np.percentile(reference, np.linspace(0, 100, bins + 1))
    breakpoints[0] = -np.inf
    breakpoints[-1] = np.inf

    ref_counts = np.histogram(reference, bins=breakpoints)[0] / len(referenc
    cur_counts = np.histogram(current, bins=breakpoints)[0] / len(current)

    # Avoid zero division
    ref_counts = np.clip(ref_counts, 1e-6, None)
    cur_counts = np.clip(cur_counts, 1e-6, None)

    psi = np.sum((cur_counts - ref_counts) * np.log(cur_counts / ref_counts)
    return psi

# Simulate: feature distribution shifts over months
np.random.seed(42)
reference = np.random.normal(50, 10, 10000)  # Training data distribution

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
psi_values = []
distributions = [reference]

for i, month in enumerate(months):
    drift = i * 2  # Gradual drift
    noise_increase = i * 0.5
    current = np.random.normal(50 + drift, 10 + noise_increase, 10000)
    distributions.append(current)
    psi = calculate_psi(reference, current)
    psi_values.append(psi)

# Visualize
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# PSI over time
colors = ['green' if p < 0.1 else 'orange' if p < 0.25 else 'red' for p in p
axes[0].bar(months, psi_values, color=colors)
axes[0].axhline(y=0.1, color='orange', linestyle='--', label='Moderate drift
axes[0].axhline(y=0.25, color='red', linestyle='--', label='Significant drif
axes[0].set_title('PSI Over Time (Data Drift Detection)')
```

```
axes[0].set_ylabel('PSI')
axes[0].legend()

# Distribution comparison
axes[1].hist(reference, bins=50, alpha=0.5, density=True, label='Reference (
axes[1].hist(distributions[-1], bins=50, alpha=0.5, density=True, label=f'{m
axes[1].set_title('Distribution Shift: Reference vs Latest')
axes[1].legend()

plt.suptitle('Data Drift Monitoring', fontsize=14)
plt.tight_layout()
plt.show()

print("PSI values:")
for month, psi in zip(months, psi_values):
    status = '✓ OK' if psi < 0.1 else '⚠ Investigate' if psi < 0.25 else '✗
    print(f"  {month}: PSI={psi:.4f} → {status}")
```

4. A/B Testing for ML Models

Don't just deploy a new model — prove it's better with statistical significance.

In [ ]:

```
# ── A/B testing for ML models ──────────────────────────────
from scipy import stats

def ab_test_models(metric_a, metric_b, alpha=0.05):
    """Compare two model variants with statistical testing."""
    # t-test
    t_stat, p_value = stats.ttest_ind(metric_a, metric_b)

    # Effect size (Cohen's d)
    pooled_std = np.sqrt((np.std(metric_a)**2 + np.std(metric_b)**2) / 2)
    cohens_d = (np.mean(metric_b) - np.mean(metric_a)) / pooled_std

    return {
        'mean_A': np.mean(metric_a),
        'mean_B': np.mean(metric_b),
        'improvement': (np.mean(metric_b) - np.mean(metric_a)) / np.mean(met
        'p_value': p_value,
        'significant': p_value < alpha,
        'cohens_d': cohens_d,
        'effect_size': 'small' if abs(cohens_d) < 0.5 else 'medium' if abs(c
    }

# Simulate: Model A (current) vs Model B (new)
np.random.seed(42)
n_users = 5000
model_a_metrics = np.random.normal(0.72, 0.08, n_users)  # Conversion rate p
model_b_metrics = np.random.normal(0.74, 0.08, n_users)  # Slightly better

results = ab_test_models(model_a_metrics, model_b_metrics)

print("A/B Test Results:")
print(f"  Model A (current): mean = {results['mean_A']:.4f}")
```

```
print(f"  Model B (new):     mean = {results['mean_B']:.4f}")
print(f"  Improvement: {results['improvement']:.2f}%")
print(f"  P-value: {results['p_value']:.6f}")
print(f"  Significant: {results['significant']} (α=0.05)")
print(f"  Cohen's d: {results['cohens_d']:.3f} ({results['effect_size']} eff

# Visualize
fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(model_a_metrics, bins=50, alpha=0.5, label=f"Model A (mean={results[
ax.hist(model_b_metrics, bins=50, alpha=0.5, label=f"Model B (mean={results[
ax.axvline(results['mean_A'], color='blue', linestyle='--')
ax.axvline(results['mean_B'], color='green', linestyle='--')
ax.set_title(f"A/B Test: p={results['p_value']:.6f} ({'Significant ✓' if res
ax.set_xlabel('Metric Value')
ax.legend()
plt.tight_layout()
plt.show()
```

5. ML Pipeline: Automating the Workflow

```
Data Validation → Training → Evaluation → Comparison →
Deployment
      ↓               ↓           ↓            ↓             ↓
  Schema check    Train model  Compute      Better than
Register +
  Drift check                  metrics      production?
Deploy
```

In [ ]:
```
# ——— Automated ML pipeline ——————————————————————————————

class MLPipeline:
    """Simple ML pipeline orchestrator."""

    def __init__(self, registry):
        self.registry = registry

    def run(self, model_name, version, train_fn, eval_fn, data):
        print(f"\n{'='*50}")
        print(f"Pipeline Run: {model_name} v{version}")
        print(f"{'='*50}")

        # Step 1: Data validation
        print("\n[1/5] Data Validation...")
        issues = self._validate_data(data)
        if issues:
            print(f"  ⚠ Issues: {issues}")
        else:
            print("  ✓ Data valid")

        # Step 2: Training
        print("\n[2/5] Training...")
        model = train_fn(data)
        print("  ✓ Model trained")
```

```
        # Step 3: Evaluation
        print("\n[3/5] Evaluation...")
        metrics = eval_fn(model, data)
        print(f"  Metrics: {metrics}")

        # Step 4: Compare with production
        print("\n[4/5] Comparing with production...")
        prod_model = self.registry.get_production_model(model_name)
        if prod_model:
            prod_acc = prod_model['metrics'].get('accuracy', 0)
            new_acc = metrics.get('accuracy', 0)
            if new_acc > prod_acc:
                print(f"  ✓ New ({new_acc:.4f}) > Production ({prod_acc:.4f}
                should_deploy = True
            else:
                print(f"  ✗ New ({new_acc:.4f}) <= Production ({prod_acc:.4f
                should_deploy = False
        else:
            print("  No production model found → auto-deploy")
            should_deploy = True

        # Step 5: Register
        print("\n[5/5] Registration...")
        self.registry.register(model_name, version, metrics)
        if should_deploy:
            self.registry.promote(model_name, version, "staging")
            print("  → Promoted to staging (run A/B test before production)"

        return metrics

    def _validate_data(self, data):
        issues = []
        if hasattr(data, 'isnull') and data.isnull().any().any():
            issues.append("missing values detected")
        if len(data) < 100:
            issues.append("dataset too small")
        return issues

# Demo
pipeline_registry = ModelRegistry()
pipeline = MLPipeline(pipeline_registry)

# Simulate
dummy_data = pd.DataFrame(np.random.randn(1000, 5), columns=[f'f{i}' for i i
train_fn = lambda data: "trained_model"
eval_fn = lambda model, data: {"accuracy": 0.87 + np.random.uniform(0, 0.05)

pipeline.run("classifier", 1, train_fn, eval_fn, dummy_data)
pipeline.run("classifier", 2, train_fn, eval_fn, dummy_data)
```

Key Takeaways

Concept	One-Line Summary
Experiment tracking	Track parameters, metrics, artifacts, code version for every run
Model registry	Version models with lifecycle: dev → staging → production → archived
Data drift (PSI)	PSI > 0.25 = significant drift, retrain the model
A/B testing	Prove new model is better with statistical significance before deploying
ML pipelines	Automate: validate → train → evaluate → compare → deploy

What to study next:

- **Notebook 18**: Deployment and serving (FastAPI, Docker, K8s)
- **Notebook 19**: System design for AI (putting it all together)